Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

Charlie Snell[◆, 1], **Jaehoon Lee**[2], **Kelvin Xu**[♣, 2] **and Aviral Kumar**[♣, 2]
[♣]Equal advising, [1]UC Berkeley, [2]Google DeepMind, [◆]Work done during an internship at Google DeepMind

arXiv:2408.03314v1 [cs.LG] 6 Aug 2024

Enabling LLMs to improve their outputs by using more test-time computation is a critical step towards building generally self-improving agents that can operate on open-ended natural language. In this paper, we study the scaling of inference-time computation in LLMs, with a focus on answering the question: *if an LLM is allowed to use a fixed but non-trivial amount of inference-time compute, how much can it improve its performance on a challenging prompt?* Answering this question has implications not only on the achievable performance of LLMs, but also on the future of LLM pretraining and how one should tradeoff inference-time and pre-training compute. Despite its importance, little research attempted to understand the scaling behaviors of various test-time inference methods. Moreover, current work largely provides negative results for a number of these strategies. In this work, we analyze two primary mechanisms to scale test-time computation: (1) searching against dense, process-based verifier reward models; and (2) updating the model's distribution over a response adaptively, given the prompt at test time. We find that in both cases, the effectiveness of different approaches to scaling test-time compute critically varies depending on the difficulty of the prompt. This observation motivates applying a "compute-optimal" scaling strategy, which acts to most effectively allocate test-time compute adaptively per prompt. Using this compute-optimal strategy, we can improve the efficiency of test-time compute scaling by more than 4× compared to a best-of-N baseline. Additionally, in a FLOPs-matched evaluation, we find that on problems where a smaller base model attains somewhat non-trivial success rates, test-time compute can be used to outperform a 14× larger model.

1. Introduction

Humans tend to think for longer on difficult problems to reliably improve their decisions [9, 17, 18]. Can we instill a similar capability into today's large language models (LLMs)? More specifically, given a challenging input query, *can we enable language models to most effectively make use of additional computation at test time so as to improve the accuracy of their response?* In theory, by applying additional computation at test time, an LLM should be able to do better than what it was trained to do. In addition, such a capability at test-time also has the potential to unlock new avenues in agentic and reasoning tasks [28, 34, 47]. For instance, if pre-trained model size can be traded off for additional computation during inference, this would enable LLM deployment in use-cases where smaller on-device models could be used in place of datacenter scale LLMs. Automating the generation of improved model outputs by using additional inference-time computation also provides a path towards a general self-improvement algorithm that can function with reduced human supervision.

Prior work studying inference-time computation provides mixed results. On the one hand, some works show that current LLMs can use test-time computation to improve their outputs [4, 8, 23, 30, 48], on the other hand, other work shows that the effectiveness of these methods on more complex tasks such as math reasoning remains highly limited [15, 37, 43], even though reasoning problems often require drawing inferences about existing knowledge as opposed to new knowledge. These sorts of conflicting findings motivate the need for a systematic analysis of different approaches for scaling test-time compute.

Corresponding author(s): csnell22@berkeley.edu

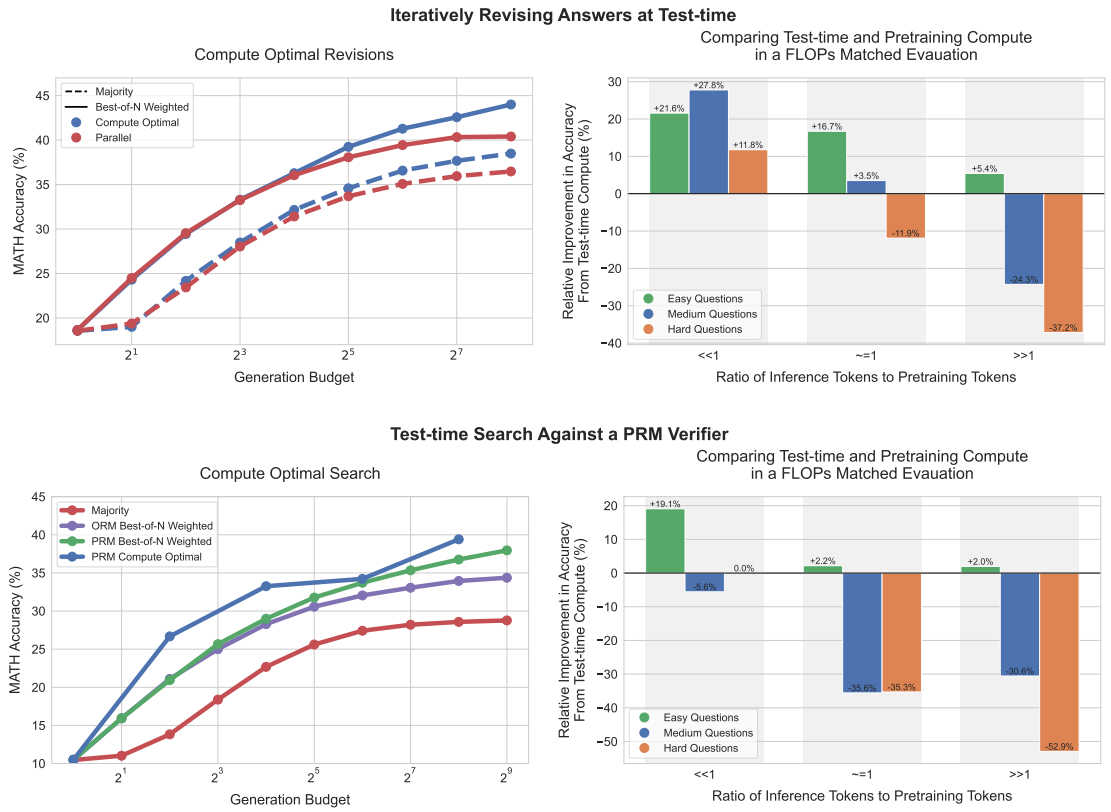

Figure 1 | ***Summary of our main results.*** **Left: Compute-optimal scaling for iterative self-refinement (i.e., revisions) and search.** On the left, we compare the compute-optimal scaling policy for our PaLM 2-S* revision model against baselines in the revision setting (top) and the PRM search setting (bottom). We see that in the revisions case, the gap between standard best-of-N (e.g. "parallel") and compute-optimal scaling gradually widens, enabling compute-optimal scaling to outperform best-of-N with 4× less test-time compute. Similarly, in the PRM search setting, we observe significant early improvements over best-of-N from compute-optimal scaling, nearly outperforming best-of-N with 4× less compute at points. See Sections 5 and 6 for details. **Right: Comparing test-time compute and model parameter scaling.** We compare the performance of compute-optimal test-time scaling with PaLM 2-S* against the performance of a ~ 14× larger pretrained model without additional test-time compute (e.g. greedy sampling). We consider the setting where we expect X tokens of pretraining for both models and Y tokens of inference. By training a larger model, we effectively multiply the FLOPs requirement for both of these terms. If we were to apply additional test-time compute with the smaller model, so as to match this larger model's FLOPs requirement, how would it compare in terms of accuracy? We see that for the revisions (top) when $Y << X$, test-time compute is often preferable to additional pretraining. However, as the inference to pretraining token ratio increases, test-time compute remains preferable on easy questions. Whereas on harder questions, pretraining is preferable in these settings. We also see a similar trend with PRM search (bottom). See Section 7 for more details.

We are interested in understanding the benefits of scaling up test-time compute. Arguably the simplest and most well-studied approach for scaling test-time computation is *best-of-N* sampling: sampling N outputs in "parallel" from a base LLM and selecting the one that scores the highest per a learned verifier or a reward model [7, 22]. However, this approach is not the only way to use test-time compute to improve LLMs. By modifying either the *proposal distribution* from which responses are obtained (for instance, by asking the base model to revise its original responses "sequentially" [28]) or by altering how the *verifier* is used (e.g. by training a process-based dense verifier [22, 45] and searching against this verifier), the ability scale test-time compute could be greatly improved, as we show in the paper.

To understand the benefits of scaling up test-time computation, we carry out experiments on the challenging MATH [13] benchmark using PaLM-2 [3] models specifically fine-tuned[1] to either revise

[1]Capability-specific finetuning is necessary to induce revision and verification capabilities into the base model on MATH

incorrect answers [28] (e.g. improving the proposal distribution; Section 6) or verify the correctness of individual steps in an answer using a process-based reward model (PRM) [22, 45] (Section 5). With both approaches, we find that the efficacy of a particular test-time compute strategy depends critically on both the nature of the specific problem at hand and the base LLM used. For example, on easier problems, for which the base LLM can already readily produce reasonable responses, allowing the model to iteratively refine its initial answer by predicting a sequence of N revisions (i.e., modifying the proposal distribution), may be a more effective use of test-time compute than sampling N independent responses in parallel. On the other hand, with more difficult problems that may require searching over many different high-level approaches to solving the problem, re-sampling new responses independently in parallel or deploying tree-search against a process-based reward model is likely a more effective way to use test-time computation. This finding illustrates **the need to deploy an adaptive "compute-optimal" strategy for scaling test-time compute**, wherein the specific approach for utilizing test-time compute is selected depending on the prompt, so as to make the best use of additional computation. We also show that a notion of *question difficulty* (Section 4) from the perspective of the base LLM can be used to predict the efficacy of test-time computation, enabling us to practically instantiate this 'compute-optimal' strategy given a prompt. By appropriately allocating test-time compute in this way, we are able to greatly improve test-time compute scaling, surpassing the performance of a best-of-N baseline while only using about **4x** less computation with both revisions and search (Sections 5 and 6).

Using our improved test-time compute scaling strategy, we then aim to understand to what extent test-time computation can effectively substitute for additional pretraining. We conduct a FLOPs-matched comparison between a smaller model with additional test-time compute and pretraining a **14x larger model**. We find that on easy and intermediate questions, and even hard questions (depending on the specific conditions on the pretraining and inference workload), additional test-time compute is often preferable to scaling pretraining. This finding suggests that rather than focusing purely on scaling pretraining, **in some settings it is be more effective to pretrain smaller models with less compute, and then apply test-time compute to improve model outputs**. That said, with the most challenging questions, we observe very little benefits from scaling up test-time compute. Instead, we find that on these questions, it is more effective to make progress by applying additional pretraining compute, demonstrating that current approaches to scaling test-time compute may not be 1-to-1 exchangeable with scaling pretraining. Overall, this suggests that even with a fairly naïve methodology, scaling up test-time computation can already serve to be more preferable to scaling up pretraining, with only more improvements to be attained as test-time strategies mature. Longer term, this hints at a future where fewer FLOPs are spent during pretraining and more FLOPs are spent at inference.

2. A Unified Perspective on Test-Time Computation: Proposer and Verifier

We first unify approaches for using test-time computation and then analyze some representative methods. First, we view the use of additional test-time compute through the lens of modifying the model's predicted distribution *adaptively* at test-time, conditioned on a given prompt. Ideally, test-time compute should modify the distribution so as to generate better outputs than naïvely sampling from the LLM itself would. In general, there are two knobs to induce modifications to an LLM's distribution: **(1) at the input level**:

since these capabilities are absent even in strong proprietary LLMs [15, 33]. However, we expect that future LLMs will be more effective at verification and revision due to both increased scale and the inclusion of additional data targeted specifically towards these capabilities [5, 24, 36]. Therefore in order to make progress towards understanding scaling of test-time computation, we must use models finetuned for these capabilities. That said, we expect future models to be pretrained for such capabilities directly, therefore avoiding the need for capability-specific finetuning.